

Desafio 1:

Código vulnerável:

```
1 import sqlite3
2
3 # Conectando ao banco de dados em memória
4 connection = sqlite3.connect(':memory:')
5 cursor = connection.cursor()
6
7 # Criando uma tabela e inserindo dados
8 cursor.execute('CREATE TABLE users (id INTEGER PRIMARY KEY, username TEXT, password TEXT)')
9 cursor.execute("INSERT INTO users (username, password) VALUES ('admin', 'admin123')")
10 cursor.execute("INSERT INTO users (username, password) VALUES ('user', 'user123')")
11 connection.commit()
12
13
14 # Função de login insegura
15 def login(username, password):
16     query = f"SELECT * FROM users WHERE username = '{username}' AND password = '{password}'"
17     cursor.execute(query)
18     return cursor.fetchone()
19
20
21 # Testando o login com SQL Injection
22 user = login(username="admin", password="' OR '1'='1")
23 print("Usuário encontrado:", user)
24 connection.close()
25
```

Run  sqlInjection x

  :

↑ Usuário encontrado: (1, 'admin', 'admin123')

Process finished with exit code 0

O código se torna vulnerável por causa que as consultas são feitas através de uma string concatenada:

```
query = f"SELECT * FROM users WHERE username = '{username}'  
AND password = '{password}'"
```

Código Corrigido:

```
1 import sqlite3
2
3 # Conectando ao banco de dados em memória
4 connection = sqlite3.connect(':memory:')
5 cursor = connection.cursor()
6
7 # Criando uma tabela e inserindo dados
8 cursor.execute('''CREATE TABLE users (id INTEGER PRIMARY KEY, username TEXT, password TEXT)''')
9 cursor.execute("INSERT INTO users (username, password) VALUES ('admin', 'admin123')")
10 cursor.execute("INSERT INTO users (username, password) VALUES ('user', 'user123')")
11 connection.commit()
12
13
14 # Função de login insegura
15 def login(username, password):
16     query = "SELECT * FROM users WHERE username = ? AND password = ?"
17     cursor.execute(query, parameters=(username, password))
18     return cursor.fetchone()
19
20
21 # Testando o login com SQL Injection
22 user = login(username="admin", password="' OR '1'='1")
23 print("Usuário encontrado:", user)
24 connection.close()
25
```

run sqlInjection x

C:\Users\S5119\AppData\Local\Microsoft\WindowsApps\python3.12.exe D:\vscode\Estudos\QA\Aula09\sqlInjection.py

Usuário encontrado: None

A correção foi feita na concatenação das Strings, trocando a concatenação por Placeholders dentro da função Login, dessa forma criamos uma camada de proteção

Código:

```
def login(username, password):
    query = "SELECT * FROM users WHERE username = ? AND password = ?"
    cursor.execute(query, (username, password))
    return cursor.fetchone()
```

Desafio 2:

Código Vulnerável:

```

import sqlite3

# Conectando ao banco de dados em memória
connection = sqlite3.connect(':memory:')
cursor = connection.cursor()

# Criando uma tabela e inserindo dados
cursor.execute('''CREATE TABLE products (id INTEGER PRIMARY KEY, name TEXT, price REAL)''')
cursor.execute("INSERT INTO products (name, price) VALUES ('Notebook', 2000.0)")
cursor.execute("INSERT INTO products (name, price) VALUES ('Smartphone', 1500.0)")
connection.commit()

# Função de busca de produtos insegura
def search_product(product_name):
    query = f"SELECT * FROM products WHERE name LIKE '%{product_name}%"
    cursor.execute(query)
    return cursor.fetchall()

# Testando a busca com SQL Injection
products = search_product("Notebook' OR '1'='1")
print("Produtos encontrados:", products)
connection.close()

```

sqlInjection2 x

:

C:\Users\55119\AppData\Local\Microsoft\WindowsApps\python3.12.exe D:\.vscode\Estudos\QA\Aula09\sqlInjection2.py
 Produtos encontrados: [(1, 'Notebook', 2000.0)]

Da mesma forma que o código acima é vulnerável por causa da query direto na String, acontece o mesmo nesse código:

```

query = f"SELECT * FROM products WHERE name LIKE
'"{product_name}%"

```

Correção:

```
import sqlite3

# Conectando ao banco de dados em memória
connection = sqlite3.connect(':memory:')
cursor = connection.cursor()

# Criando uma tabela e inserindo dados
cursor.execute('''CREATE TABLE products (id INTEGER PRIMARY KEY, name TEXT, price REAL)''')
cursor.execute("INSERT INTO products (name, price) VALUES ('Notebook', 2000.0)")
cursor.execute("INSERT INTO products (name, price) VALUES ('Smartphone', 1500.0)")
connection.commit()

# Função de busca de produtos insegura
def search_product(product_name):
    query = f"SELECT * FROM products WHERE name LIKE ?"
    cursor.execute(query, parameters: ("% " + product_name + "%",))
    return cursor.fetchall()

# Testando a busca com SQL Injection
products = search_product("Notebook' OR '1'='1")
print("Produtos encontrados:", products)
connection.close()
```

sqlInjection2 x

...

C:\Users\55119\AppData\Local\Microsoft\WindowsApps\python3.12.exe D:\.vscode\Estudos\QA\Aula09\sqlInjection2.py

Produtos encontrados: []

Corrigindo com o Placeholder, criamos uma camada de segurança a mais para a query

Código:

```
query = f"SELECT * FROM products WHERE name LIKE ?"
cursor.execute(query, ("% " + product_name + "%",))
```